Thunk
Wiki: https://wiki.haskell.org/Thunk

Boxed/Unboxed type
Lifted/Unlifted type
[Wiki] Boxed/unboxed: https://wiki.haskell.org/Unboxed_type
GHC unboxed types and primitives: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#

There are 3 types of values in Haskell:

	Examples	**Boxed/Unboxed** raw bytes (unboxed) pointer (boxed)	**Lifted/Unlifted** primitive (unlifted) non-primitive (lifted)
primitive value i.e. **raw bytes** in memory Analogous to primitive **scalars** in C e.g. C long int	Int# ~ C long int Double# ~ C double Char# ~ C char (# a, b #)	Unboxed	-
primitive container i.e. raw **pointer** for a primitive value (in **heap memory**) e.g. primitive array Analogous to an array in C	Array# a ~ C array (pointer) Array# Int#	Boxed	Unlifted
normal Haskell type All other pointers i.e. pointers for non-primitives	data Int = I# Int# data Char = C# Char# data Maybe	Boxed	Lifted

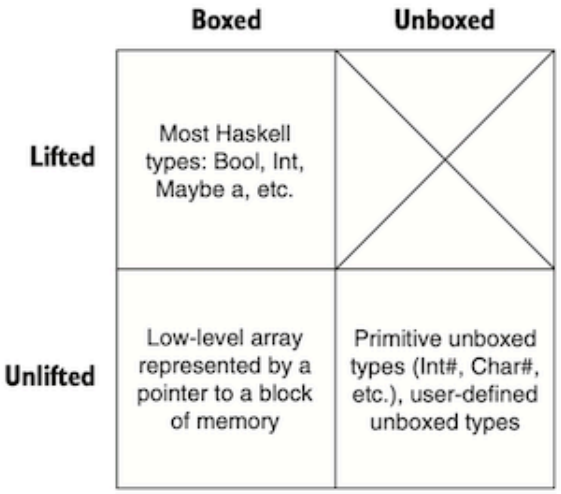


Figure 9.7 Relations
boxed/unboxed types

Boxed/Unboxed type
Wiki: https://wiki.haskell.org/Unboxed_type
GHC unboxed types and primitives: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#
Whether the value is a raw **primitive value (unboxed)** in memory or a **pointer (boxed)**.

a) **unboxed type (aka primitive value type)**:
A **primitive value** represented by **raw bytes** directly in memory
i.e. same as C base types
e.g.

unboxed type		C
Int#	=	long int
Double#	=	double
Char#	=	char

Primitive types have **primitive values** and **primitive operators**
i.e. primitive value = raw C value
primitive operator = raw C operator

Most primitive values are suffixed by the **hash symbol #** but by **convention** only
Primitive types are defined in **GHC.Exts**
Ref: https://hackage-content.haskell.org/package/base/docs/GHC-Exts.html
Primitive value are usually hidden and require the **GHC MagicHash** extension
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/magic_hash.html
e.g.

```
{-# LANGUAGE MagicHash #-}
import GHC.Exts

-- unboxed types
Addr#           -- same as C pointer
Int#            -- same as C long Int
Char#           -- C char
Double#
```

```
-- primitive values
1# :: Int#         -- primitive integer 1 i.e. (int) 1
in C
'c'# :: Char#      -- (char) 'c'
1.1# :: Double#    -- (double) 1.1

-- unboxed tuple
(# 1#, 2# #)       -- tuple of primite
(# 1, 2 #)         -- tuple of non-primitive

-- primitive operations
(+#) :: Int# -> Int# -> Int#        -- + operator in C
(*#) :: Int# -> Int# -> Int#        -- * operator in C
```

b) **boxed type**: a pointer

e.g. int* in C

Almost all standard user-facing values are boxed:
e.g.

```
-- literals are boxed
1                     -- literal 1
'c'
1.1

-- data constructors are boxed
True                  -- i.e. Bool value
Just 1
Either 1
[1]

-- functions and operators are boxed
id                    -- i.e. function
const
(+)                   -- (+) operator

-- type constructors themselves are boxed
Int                   -- i.e. Int type
Char
Maybe
[]
```

Builtin types (Int, Char, Double, etc)
Builtin boxed types are wrappers of unboxed types
e.g.

```
{-# LANGUAGE MagicHash #-}

--  defined in GHC.Exts
data Int = I# Int#        -- I# = data constructor of
Int
```

```
                          -- Int# = field of Int i.e. an
unboxed type
data Char = C# Char#      -- C# = data constructor of
Char
                          -- Char# = field of Char i.e.
a primitive type

-- e.g. Int
x = 1 :: Int
x = I# 1#                 -- same
                          -- 1# :: Int# = primitive int
value = (int) 1 in C
x = I# 1# :: Int
x = I# (1# :: Int#)
1 == I# 1#                -- True

-- e.g. Char
x = 'a' :: Char
x = C# 'a'#               -- same
                          -- 'a'# = primitive char
value = (char) 'a' in C
x = C# 'a'# :: Char
x = C# ('a'# :: Char#)
'a' == C# 'a'#            -- True

-- e.g. (+) operator
x = 1 + 2
x = I# (1# +# 2#)         -- same
                          -- +# :: Int# -> Int# -> Int#
= primitive operator = (+) operator in C
3 == I# (1# +# 2#)        -- True
```

Lifted/unlifted
GHC unboxed types and primitives: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#
Whether a **pointer** is for a **primitive** or **non-primitive** value.

unlifted	pointer for a **primitive** type e.g. a primitive array Array#
lifted	pointer for a **non-primitive** type i.e. all normal Haskell values

In implementation, unlifted pointers can **NOT be a thunk**
i.e. it is not evaluated lazily

Corollary: only a **lifted value** can be a **bottom** (undefined).
e.g.

```
-- Int is lifted
let x = undefined :: Int in ()     -- OK. No error as
long as x is not used
```

```
-- Int# is NOT lifted
let x = undefined :: Int# in ()    -- ERROR even if x is
not used.
```

e.g.

```
import GHC.Prim

-- unboxed types
let x = undefined :: Int# in ()           -- ERROR
Unboxed types are never lifted
let x = undefined :: Char# in ()          -- ERROR

-- boxed, lifted types
let x = undefined :: Int in ()            -- OK. Int is
a lifted type
let x = undefined :: Maybe a in ()        -- Maybe a
let x = undefined :: [a] in ()            -- list
let x = undefined :: (a->b) in ()         -- function
let x = undefined :: Num a => a in ()     -- class
let x = undefined :: a in ()              -- any boxed
type

-- boxed, unlifted types
let x = undefined :: Array# Int# in ()    -- ERROR.
Array# is unlifted
let x = undefined :: RealWorld in ()      -- ERROR
```

==========
======
==OLD==

OLD

Strict function/expression
Strictness
Normal form
Strict evaluation: https://www.haskell.org/onlinereport/haskell2010/haskellch6.html#x13-1260006.2
[wiwinwlh] strictness: https://sdiehl.github.io/wiwinwlh/#strictness

i.e. a strict function/expression in a **non-strict semantics** language
In a non-strict language, a **function** cannot ensure **total strictness**
i.e. strict in **every level**
i.e. in **normal form** = every subexpression has been evaluated.
A function can only ensure strictness only on **one level**
e.g.

```
If
    y = f x            x = g1 (g2 (g3 ... (gn x0) ...)))
then
    !y = f! x
       = f! (g1 (g2 ... (gn x0) ...))            Only
strictness of f
                                                  NOT
total strictness of argument
NOT
    !y = f! (g1! (g2! ... (gn! !x0) ... ))       Normal
form
```

Hence the **definition**

f is strict <=> f undefined = undefined i.e. f undefined = ERROR

e.g. strict functions

```
-- f undefined = undefined
x = id undefined              -- ERROR. i.e. id is
strict
x = head undefined            -- ERROR.
x = length undefined          -- ERROR
x = null undefined            -- ERROR
x = fromJust undefined        -- ERROR


-- e.g. f [undefined] is OK
x = id [undefined]            -- [undefined] i.e. no
error
x = head [undefined]          -- OK
x = length [undefined]        -- 1
x = null [undefined]          -- FALSE

-- e.g. f (Just undefined) is OK
x = id (Just undefined)         -- Just undefined
x = fromJust undefined          -- OK
```

e.g. foldr'

```
-- foldr is non-strict
head $ foldr (++) "" ["a",undefined]      -- OK. 'a'

-- foldr' is strict
-- i.e. error even if the value is directly not used
head $ foldr' (++) "" ["a",undefined]     -- ERROR
```

Strictness in Haskell

Strict in argument
Strict in result
[FPComplete] all about strictness: https://academy.fpblock.com/haskell/tutorial/all-about-strictness/
Is id strict?

```
id undefined              -- undefined
id (1 + undefined)        -- 1 + undefined NOT in WHNF
```

e.g. different strictnesses

```
{-# LANGUAGE BangPatterns #-}

f :: a -> b                       -- non-strict?
f x = g x

f' :: a -> b                      -- strict in argument
x
f' !x = g x

f'' x :: a -> b                   -- strict in result up
to WHNF
f'' x = let !y = g x in y

f''' x :: a -> b                  -- strict in result up
to WHNF + argument
f''' !x = let !y = g x in y
```

strict != performant, even with deepseq

```
import Control.Deepseq

sum :: [Int] -> Int
sum l = let !y = foldl (+) l 0 in y        -- BAD:
strict in result, but still slow

sum :: [Int] -> Int                        -- BAD:
deepseq still doesn't help
sum l = force $ foldl (+) l 0 in y

sum' :: [Int] -> Int                       -- GOOD
sum' l = foldl' (+) l 0
```

evaluation (laziness)
Strict evaluation
Strict evaluation: https://www.haskell.org/onlinereport/haskell2010/haskellch6.html#x13-1260006.2
[wiwinwlh] laziness: https://sdiehl.github.io/wiwinwlh/#laziness
non-strict semantics implies that **pure expressions** are NEVER **evaluated** by default.

non-strict semantics implies that **pure expressions** are NEVER **evaluated** by default.

In theory, the only way to force evaluation is during **execution**
i.e. in Haskell, an **IO expression**
e.g.

```
print x
```

Put another way, **pure expressions** NEVER trigger an **error**

```
x :: Int
x = undefined         -- no error in pure expression

y = Int
y = x + 1             -- no error in pure expression

test = do
  print y             -- non-pure
```

Lazy evaluation has 2 important implications:
1) **performance**: thunks are created, which can negatively impact performance. See below.
2) **error:** errors cannot be **handled** by pure expressions

e.g.

```
-- e.g. WHNF of Int
4 :: Int                  -- WHNF
2 * 2 :: Int              -- NO
2 * undefined :: Int      -- NO
undefined * 2 :: Int      -- NO
2 * (read @Int "2")       -- NO

-- e.g. WHNF of Maybe
Just 1                    -- WHNF
Just undefined            -- WHNF
undefined <$> Just 1      -- NO
Just 1 >>= Just 2         -- NO


-- e.g. GOTCHA: WHNF of newtype like Identity
Identity 1                -- WHNF
1                         -- same WHNF
Identity undefined        -- WHNF
undefined                 -- same WHNF


-- e.g. WHNF of list
undefined                 -- WHNF
[undefined]               -- WHNF i.e. undefined:[]
undefined:[]              -- WHNF
undefined:undefined       -- WHNF i.e.
undefined:undefined
[1] ++ undefined          -- WHNF i.e. 1:undefined
[] ++ undefined           -- NO since it needs to
```

```
determine x:... = undefined
undefined ++ [1]          -- NO since it needs to
determine x:...:[1] = undefined:[1]
```

Boxed/Unboxed type
Wiki: https://wiki.haskell.org/Unboxed_type
GHC unboxed types and primitives: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#
Haskell values are either
unboxed (aka **primitive type)** **raw value** directly in **memory**,
i.e. same as C **base types in C** (int, double, char, etc)
boxed **pointer** to a value in **heap memory**
i.e. same as **C pointers**

Boxed vs unboxed
a) **unboxed type (primitive type)**
i.e. same as C base types
e.g.

unboxed type		C
Int#	=	long int
Double#	=	double
Char#	=	char

Primitive types have **primitive values** and **primitive operators**
i.e. primitive value = raw C value
primitive operator = raw C operator

Most primitive values are suffixed by the **hash symbol #** but by **convention** only
Primitive types are defined in **GHC.Exts**
Ref: https://downloads.haskell.org/ghc/latest/docs/libraries/base-4.17.0.0/GHC-Exts.html
Primitive value are usually hidden and require the **GHC MagicHash** extension
Ref: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/magic_hash.html
e.g.

```
{-# LANGUAGE MagicHash #-}
import GHC.Exts

-- unboxed types
Addr#           -- same as C pointer
Int#            -- same as C long Int
Char#           -- C char
Double#

-- primitive values
1# :: Int#         -- primitive integer 1 i.e. (int) 1
in C
'c'# :: Char#      -- (char) 'c'
1.1# :: Double#    -- (double) 1.1

-- primitive operations
(+#) :: Int# -> Int# -> Int#        -- + operator in C
(*#) :: Int# -> Int# -> Int#        -- * operator in C
```

b) **boxed type**
i.e. same as C pointer
  e.g. int* in C

Almost all standard user-facing values are boxed:
e.g.

```
-- literals are boxed
1                    -- literal 1
'c'
1.1

-- data constructors are boxed
True                 -- i.e. Bool value
Just 1
Either 1
[1]

-- functions and operators are boxed
id                   -- i.e. function
const
(+)                  -- (+) operator

-- type constructors themselves are boxed
Int                  -- i.e. Int type
Char
Maybe
[]
```

Builtin types (Int, Char, Double, etc)
Builtin boxed types are wrappers of unboxed types
e.g.

```
{-# LANGUAGE MagicHash #-}

--  defined in GHC.Exts
data Int = I# Int#        -- I# = data constructor of
Int
                          -- Int# = field of Int i.e. an
unboxed type
data Char = C# Char#      -- C# = data constructor of
Char
                          -- Char# = field of Char i.e.
a primitive type

-- e.g. Int
x = 1 :: Int
x = I# 1#                 -- same
                          -- 1# :: Int# = primitive int
value = (int) 1 in C
x = I# 1# :: Int
```

```
x = I# (1# :: Int#)
1 == I# 1#                -- True

-- e.g. Char
x = 'a' :: Char
x = C# 'a'#               -- same
                          -- 'a'# = primitive char
value = (char) 'a' in C
x = C# 'a'# :: Char
x = C# ('a'# :: Char#)
'a' == C# 'a'#            -- True

-- e.g. (+) operator
x = 1 + 2
x = I# (1# +# 2#)         -- same
                          -- +# :: Int# -> Int# -> Int#
= primitive operator = (+) operator in C
3 == I# (1# +# 2#)        -- True
```

Lifted/unlifted type
Unlifted type
GHC unboxed types and primitives: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html#
A type is **lifted type** if its evaluation can be **delayed**.
i.e. can be assigned an **undefined**.
e.g.

```
-- Int is lifted
let x = undefined :: Int in ()     -- OK. No error as
long as x is not used

-- Int# is NOT lifted
let x = undefined :: Int# in ()    -- ERROR even if x is
not used.
```

Lifted/unlifted vs **boxed/unboxed**
Only **pointers** can be **lazy**, and hence lifted.
Raw values in memory cannot be lazy, and hence cannot be lifted.
i.e.
All **unboxed** types are unlifted
Only **boxed** types may be lifted
In particular, most **user-facing** types (boxed) are lifted
e.g.

```
import GHC.Prim

-- boxed, lifted types
let x = undefined :: Int in ()         -- OK. Int is a
lifted type
let x = undefined :: Maybe a in ()     -- Maybe a
let x = undefined :: [a] in ()         -- list
```

```
let x = undefined :: (a->b) in ()      -- function
let x = undefined :: Num a => a in ()  -- class
let x = undefined :: a in ()           -- any boxed type

-- unboxed, unlifted types
let x = undefined :: Int# in ()        -- ERROR. Int# is
unlifted
let x = undefined :: Char# in ()

-- boxed, unlifted types
let x = undefined :: Array# Int in ()  -- ERROR. Array#
is unlifted
let x = undefined :: RealWorld in ()
```